Day 2 exercises

Day 2 Exercises: Programming Fundamentals

PhD Course in Integrative Neurosciences - Introduction to Scientific Programming

Before You Begin

Before starting these exercises, open the handout for Day 2 and familiarize yourself with its contents - it contains essential information for completing these exercises. Exercises marked with an asterisk (*) require more thought and experimentation.

> [!tip] **Tips for Success:**
>
> - Test your code frequently as you write it
> - Use meaningful variable names
> - Add comments to explain your logic
> - Save your work in appropriately named files (e.g., `ex1.py`, `ex2.py`)
> - Don't hesitate to use `help()` or `dir()` to explore Python's capabilities
> - Search online for documentation, but try not to use LLMs; it is important to understand these principles

---

Part 0: Setting up and running Python

These exercises are the checkpoint of the first hour. Work in pairs: one person types, the other reads the output and error messages, then swap. The scripts are in the `setup/` folder; the guide is `setup/setup_tutorial.md`.

Exercise 0.1: *Find your way around the terminal*

Open **Anaconda Prompt** (Windows) or **Terminal** (macOS, Linux).

1. Print the current folder. What is it?
2. List the files in it.
3. Go to the course folder you downloaded, then into `day02-fundamentals/setup`. List the files there. How many `.py` files do you see, and what are they called?
4. Create a folder called `my_work`, go into it, then go back up one folder.
5. Which command shows which Python the terminal would use? What does it print?

Exercise 0.2: *Environment and running code four ways*

1. Create and activate the environment (`environment-base.yml`, see the guide). How do you know it is active?
2. From the course folder run `python day02-fundamentals/setup/check_python.py`. Does it end with `RESULT: ... ready`? If a line says `WARN` or `FAIL`, fix it with the guide or ask for help.
3. **Interactive prompt.** Start `python`. Compute `2 + 3`. Create a variable `name` with your name and print a greeting using an f-string. Leave with `exit()`. Then try `ipython` and repeat. What differs?
4. **Script from the terminal.** Go into `day02-fundamentals/setup` and run `python hello.py`.
5. **Wrong folder on purpose.** Go back to the course folder and run `python hello.py`. What error do you get and why? Fix it with the right command.
6. **VS Code.** Open the course folder in VS Code, open `hello.py`, select the interpreter, and run it with the play button. Where does the output appear?
7. Edit `hello.py` so it also asks for your favourite brain region and prints a sentence with it. Run it again, from VS Code and from the terminal.

Exercise 0.3: *Fix the broken scripts*

The four scripts `setup/broken_1.py` to `broken_4.py` each contain one mistake. For each script: run it, **read the error from the bottom**, say what type of error it is and what it means, then fix it and run it again. Notes for broken_4: first run it from the main course folder (`python day02-fundamentals/setup/broken_4.py`), then think about where Python looks for `recording_notes.txt`.

Script	Error type you see	What was wrong
`broken_1.py`		
`broken_2.py`		
`broken_3.py`		
`broken_4.py`		

Exercise 0.4: *Checkpoint*

Tick all five. If one fails after 10 minutes, raise your hand or switch to Plan B (Google Colab) and we will fix your computer after class.

- ☐ `python --version` works in the terminal
- ☐ the environment `scientific-programming` is active (its name is in the prompt)
- ☐ `check_python.py` ends with `RESULT: ... ready`
- ☐ `hello.py` runs from the terminal and from VS Code
- ☐ the four broken scripts are fixed

Part 1: Variables, Data Types, and Basic I/O

Exercise 1: *Value and Type of Expressions*

Using your Python interpreter (interactive mode or a script), check the value and type of the following expressions and determine which cause errors.

First, define these variables:

```
i = 10
j = 3
f = 3.0
c = 4.0 + 3.5j
s = 'hello'
```

Now evaluate each of the following expressions. For each one:

- Predict what will happen
- Execute the code
- Check the result and its type using `type()`
- Explain why you got that result

Expressions to test:

```
2 * i
i + f
s + ' world'
s + i
s + str(i)
i / j
i / float(j)
i / f
i // j
i // f
c * f
f ** 2
i ** 0.5
```

> [!question] **Questions:**
>
> 1. What happens when you combine an `int` and a `float` in arithmetic operations? What type is the result? Is this sensible?
> 2. What's the difference between `/` (true division) and `//` (floor division)?
> 3. What happens when you try to add a string and a number? How can you fix this?
> 4. Can you perform arithmetic with complex numbers? What operations work?

Exercise 2: *Understanding Variables and Identity*

Part a) Same or Different?

Examine these two programs carefully. What is the difference in their behavior, and why does it happen? **Program 1:**

```
x = 'ab'
y = x
x = 'ac'
print(y)
```

Program 2:

```
x = ['a', 'b']
y = x
x[1] = 'c'
print(y)
```

> [!question] **Questions:**
>
> 1. What does each program print?
> 2. Why do they behave differently?
> 3. What does this tell you about how Python handles strings vs. lists?
> 4. How would you modify Program 2 to make `y` independent of `x`?

Hint: Use `id()` to check object identity, and read about mutable vs. immutable types.

Part b) Swapping Values

Write a Python program that swaps the values of two variables. For example:

```
a = 10
b = 20
# TODO Your code here
# After your code runs:
# a should be 20
# b should be 10
```

Try to find at least two different ways to accomplish this swap.

Exercise 3: *List and String Slicing*

Save your solution in a file named `ex3.py`

Define the following variables:

```
mystring = '012345678'
mylist = ['0', '1', '2', '3', '4', '5', '6', '7', '8']
```

Use slicing to perform the following operations. For each, print the result to verify it worked.

1. Set variable `a` to be the first five characters of `mystring`
2. Set variable `b` to be everything except the first five characters of `mystring`
3. Set variable `c` to be everything from the second character until the next-to-last character of `mystring`
4. Set the 3rd item in `mylist` to the last value of `mystring`, then print `mylist`
5. Try to set the 3rd item in `mystring` to the last value of `mylist`, then print `mystring`. **What happens? Why?** How can you achieve the same result?
6. Set a variable `d` to be every second item of `mylist`
7. Set a variable `e` to be every item of `mylist` in reverse order
8. **Challenge:** Extract the middle three items from `mylist` using slicing

> [!question] **Questions to consider:**
>
> - Why can you modify list elements but not string characters?
> - What's the difference between mutable and immutable types?
> - How does negative indexing work?

Exercise 4: *Input and Output*

Part a) Writing to Screen

Save as `ex4a.py`

Write a Python program that:

1. Prints your name
2. Prints your research interest or favorite scientific topic
3. Uses f-strings to create a formatted message combining both

Example output:

```
Name: Ana Silva
Research interest: Synaptic plasticity in hippocampal neurons
Hello! I'm Ana Silva and I study synaptic plasticity in hippocampal neurons.
```

Part b) Reading Input

Save as `ex4b.py`

Write a program that:

1. Asks the user for their first name
2. Asks for their last name
3. Prints a personalized greeting

Example interaction:

```
Enter your first name: Carlos
Enter your last name: Santos
Hello, Carlos Santos! Welcome to the scientific programming course.
```

Part c) Reading Numeric Input

Save as `ex4c.py`

Write a program that:

1. Asks the user for their age
2. Converts the input to an integer
3. Calculates how many years until they turn 100
4. Prints an appropriate message

Challenge: Add error handling for non-numeric input.

Exercise 5: *Temperature Conversion*

Save as `ex5.py`

Write a program that asks the user for a temperature in Celsius and converts it to Fahrenheit.

Formula: °F = (°C × 9/5) + 32

Example interaction:

```
Enter temperature in Celsius: 37
37.0°C is equal to 98.6°F
```

Extensions:

1. Format the output to 1 decimal place
2. Allow the user to choose which direction to convert (C to F or F to C)
3. Add input validation to ensure the temperature is reasonable (e.g., above absolute zero: -273.15°C)

Exercise 6: *Discovering Python's Capabilities*

Part a) Exploring Lists

Save as `ex6a.py`

Start by defining a non-empty list of your choice (e.g., neuron types, brain regions, or experimental conditions).

Your task is to discover and use the following list methods **without looking them up** (use `dir()` and `help()`):

1. Add an element to the end of the list
2. Remove the last element you added
3. Remove the first element from the list
4. Insert an element at a specific position
5. Find the index of a specific element
6. Count how many times an element appears
7. Sort the list

Example starting point:

```
brain_regions = ["cortex", "hippocampus", "cerebellum"]

# Use dir(brain_regions) to see available methods
# Use help(brain_regions.append) to learn about specific methods
```

Bonus: What's the difference between `sort()` and `sorted()`?

Part b) Exploring Strings

Save as `ex6b.py`

Write a program that:

1. Reads the user's first name and last name (as a single input: "firstname lastname")
2. Reverses the order to "lastname, firstname"

Discover and use string methods for:

- Splitting a string into parts
- Joining strings together
- Capitalizing text

Example interaction:

```
Enter your full name: maria santos
Output: Santos, Maria
```

Extensions:

- Handle middle names
- Handle unusual capitalization in input
- Discover what `strip()`, `upper()`, `lower()`, and `title()` do

Part 2: Operators, Expressions, and Control Structures

Exercise 7: *Arithmetic Operations on Different Types*

Save as `ex7.py`

Part a) Type Mixing

Write a program that tests what happens when you combine different numeric types:

```
a = 2       # int
b = 3.75    # float

# Test these operations:
c = a + b
d = a - b
e = a * b
f = a / b
g = a // b
h = a % b
i = a ** b

# For each result, print:
# 1. The result value
# 2. The result type
# 3. Whether it makes sense
```

> [!question] **Questions:**
>
> - What type does each operation return?
> - Is Python's behavior sensible?
> - When might you want to explicitly convert types?

Part b) Division Behavior

Test the division operator's behavior with different types:

```
# Test these:
10 / 3       # float division
10 // 3      # floor division (int)
10.0 // 3.0  # floor division (float)
10 % 3       # modulo (remainder)
```

> [!question] **Questions:**
>
> - What's the difference between `/` and `//`?
> - Why does `10.0 // 3.0` return a float?
> - When would you use `%` (modulo)?

Part c) Modulo Applications

The modulo operator (`%`) returns the remainder of division.

Write code to:

1. Check if a number is even or odd
2. Check if number `a` is a factor of number `b` (divides evenly)
3. Extract the last digit of a number
4. Convert seconds to minutes and seconds

Example for task 4:

```
total_seconds = 185
minutes = ?
seconds = ?
# Should give: 3 minutes and 5 seconds
```

Exercise 8: *Logical Expressions and Comparisons*

Save as `ex8.py`

Part a) Boolean Logic

Predict the result of these expressions, then verify:

```
x = 5
y = 10
z = 5

# Predict then test:
x < y
x == z
x <= z
y != z
x < y and y > z
x < y or y < z
not (x == z)
x < y < 15
```

Part b) Chained Comparisons

Python allows chained comparisons. Test these:

```
age = 25

# Traditional way:
result1 = age >= 18 and age < 30

# Chained comparison:
result2 = 18 <= age < 30

# Test with different ages:
# 15, 18, 25, 30, 35
```

Part c) Neuroscience Application

Write expressions to check if:

1. A membrane potential is in the depolarization range (-55 mV to -30 mV)
2. A firing rate is abnormally high (>100 Hz) or low (<1 Hz)

3. An experiment is valid (has consent AND age >= 18 AND no exclusion criteria)

```
voltage = -45
firing_rate = 12.5
has_consent = True
age = 22
has_exclusion = False

# Write boolean expressions for each condition
```

Exercise 9: *Warming Up with Loops*

Part a) For-Loop Sum

Save as `ex9a.py`

Write a program with a for-loop that adds up all whole numbers between 1 and 100 and prints the total.

Expected output: 5050

Extension: Verify your result using the formula: sum = n(n+1)/2 where n=100

Part b) While-Loop Sum

Save as `ex9b.py`

Write a program with a while-loop that:

1. Starts at 1 and adds consecutive numbers
2. Stops when the total exceeds 1000
3. Prints the number at which this happens and the final total

Extension: How many numbers did it take? Print this information.

Part c) Nested Loops

Save as `ex9c.py`

Write a program using nested loops to create a multiplication table from 1 to 10.

Example output:

```
1 x 1 = 1    1 x 2 = 2    1 x 3 = 3    ...
2 x 1 = 2    2 x 2 = 4    2 x 3 = 6    ...
...
```

Extension: Format the output nicely with aligned columns.

Exercise 10: *Function Analysis with Loops*

Save as `ex10.py`

Note: No imports are needed. `range()` only works with integers, so to build a grid of floats loop over an integer counter `i` and compute `x = start + i * step`. Round the result (`round(x, 1)`) to avoid values like `0.30000000000000004`.

Part a) Calculate and Display a Function

Write a program that:

1. Calculates $y = x^2 - 2x$ from x = -5 to x = 5 in steps of 0.1
2. Stores x values in one list and y values in another
3. Prints a table of x and y (every 10th point is enough) using f-strings with aligned columns
4. Draws a simple text plot: one line per x value, with a bar of `#` characters whose length is proportional to y (ignore negative values or mark them with `-`)

Hints:

- Use `for i in range(101)` and `x = round(-5.0 + i * 0.1, 1)` for x values
- Create an empty list for y values
- Use a for-loop to calculate each y value
- Append y values to your list
- Bar length: `int(y / 2)` characters gives a compact plot, `"#" * length`

Part b) Finding a Root

A **root** of an equation y = f(x) is a value of x where y = 0.

Modify your program from part a to find and print the first root (where y changes from positive to negative or vice versa).

Approach:

- Loop through your y values
- Check if consecutive values have opposite signs
- Print the corresponding x value

Hint: You'll need to compare `y_values[i]` with `y_values[i-1]`

Part c) Finding All Roots (*)

Adapt your solution to find **all** roots in the range, not just the first one.

Extension: Print both roots and indicate whether each is a crossing from positive-to-negative or negative-to-positive.

Part d) Calculating the Derivative (*)

The derivative of a function can be approximated by:

$$\frac{dy}{dx} \approx \frac{\Delta y}{\Delta x} = \frac{(y[i+1] - y[i])}{(x[i+1] - x[i])}$$

Add code to:

1. Calculate the derivative at each point
2. Print a table with x, y and the derivative side by side (every 10th point)
3. Observe where the derivative crosses zero - what does this mean?

Hint: The derivative list will have one fewer element than the original function.

Exercise 11: *Area Under a Curve (Numerical Integration. *)*

Save as `ex11.py`

We'll approximate the area under a curve by dividing it into rectangles - this is called the **Riemann sum**.

Start with:

```
import math
```

Part a) Visualizing the Riemann Sum

Write a program that:

1. Calculates y = exp(x) (use `math.exp`) from x = 0 to x = 4 with step size 0.5
2. Prints one line per rectangle with its x position, its height and its area (width × height)
3. Displays each rectangle as a text bar: a line of **#** characters whose length is proportional to its height

The bars should show how the rectangles follow the curve.

Hint:

```
step = 0.5
n = int(4.0 / step)              # number of rectangles
for i in range(n):
    x = i * step
    height = math.exp(x)
    print(f"x = {x:4.1f}  {'#' * int(height)}")
```

Part b) Calculating the Total Area

Modify your program to calculate the total area of all rectangles, now with step size 0.1.

Formula: Total area = Σ (height × width) = Σ (y_i × Δx)

Use a for-loop to sum up the areas of all rectangles and print the result. Compare it with the exact value `math.exp(4) - math.exp(0)`, and print the error.

Note: This approximates the following integral: $\int_0^4 \exp(x)\,dx$

Extension: Repeat the calculation for 10, 100, 1000 and 10000 rectangles, and for the left (`x = a + i*step`), right (`x = a + (i+1)*step`) and midpoint (`x = a + (i+0.5)*step`) choices of the sample point. Print a table with the approximation, the exact value and the error. How does the error shrink as the number of rectangles grows?

Part c) Finding the Upper Limit

Write a program that adds up rectangles under y = exp(x) until the total area exceeds 25.

Print:

1. The x value where this happens
2. The total area

Hint: Use a while-loop instead of a for-loop.

Part d) Alternative Implementations (*)

You probably used a for-loop for part b and a while-loop for part c.

Challenge:

- Implement part b using a while-loop
- Implement part c using a for-loop with `break`

Which approach is more natural for each problem? Why?

What you just did:

1. You approximated the integral:
$$\int_0^a \exp(x)\,dx \approx \sum \exp(x)\Delta x$$
where $\Delta x = 0.1$ and $a = 4.0$ (or the upper limit you found).
2. In part c, you found the value of `a` such that $\int_0^a \exp(x)\,dx = 25$.

This is a fundamental technique in numerical analysis!

Exercise 12: *Control Flow Mastery*

Save as `ex12.py`

Part a) Grade Calculator

Write a program that:

1. Asks the user for a numerical score (0-100)
2. Prints the corresponding letter grade using this scale:
    - A: 90-100
    - B: 80-89
    - C: 70-79
    - D: 60-69
    - F: 0-59

Use if-elif-else statements.

Part b) Spike Train Classifier

Write a program that classifies neural firing patterns:

```
firing_rate = float(input("Enter firing rate (Hz): "))

# Classify as:
# "Silent" if rate < 1
# "Low" if 1 <= rate < 10
# "Moderate" if 10 <= rate < 50
# "High" if 50 <= rate < 100
# "Hyperactive" if rate >= 100
```

Part c) Password Validator (*)

Write a program that validates a password must meet these criteria:

- At least 8 characters long
- Contains at least one digit
- Contains at least one uppercase letter

Give the user 3 attempts. If they succeed, print "Password accepted". If they fail all attempts, print "Too many failed attempts".

> [!hint] **Hints:**
>
> - Use string methods: `isdigit()`, `isupper()`, `islower()`
> - Use `any()` function with a loop or list comprehension
> - Use a for-loop with a counter or a while-loop

Exercise 13: *Exception Handling*

Save as `ex13.py`

Part a) Safe Division

Write a program that:

1. Asks for two numbers
2. Divides the first by the second
3. Handles both ValueError (non-numeric input) and ZeroDivisionError

Example interaction:

```
Enter numerator: 10
Enter denominator: 0
Error: Cannot divide by zero!

Enter numerator: 10
```

```
Enter denominator: abc
Error: Please enter a valid number!

Enter numerator: 10
Enter denominator: 2
Result: 5.0
```

Part b) Robust Age Input

Write a program that repeatedly asks for the user's age until valid input is received:

- Must be a number
- Must be positive
- Must be less than 120

Use try-except with a while loop.

Part c) File-Safe Calculator (*)

Create a calculator that:

1. Performs basic operations (+, -, *, /)
2. Handles all possible errors gracefully
3. Keeps running until user types "quit"

Example:

```
Enter calculation (or 'quit'): 10 + 5
Result: 15.0

Enter calculation (or 'quit'): 10 / 0
Error: Cannot divide by zero!

Enter calculation (or 'quit'): 10 * abc
Error: Invalid number!

Enter calculation (or 'quit'): quit
Goodbye!
```

Part 3: Functions and Modular Programming

Exercise 14: *Function Fundamentals*

Part a) From Loop to Function

Step 1: Write a program with a for-loop that adds up the squares of all whole numbers between 1 and 20.

```
        # TODO Your for-loop here
```

Step 2: Turn your answer into a function called `sum_of_squares()` that returns the sum of squares from 1 to 20.

```
def sum_of_squares():
    # TODO Your code here
    pass

result = sum_of_squares()
print(f"Sum of squares from 1 to 20: {result}")
```

Step 3: Modify your function to accept a parameter `n` and calculate the sum of squares from 1 to n.

```
def sum_of_squares(n):
    # TODO Your code here
    pass

print(sum_of_squares(20))   # Should give same result as before
print(sum_of_squares(10))   # Should give 385
```

Part b) While-Loop Function

Step 1: Write a program with a while-loop that adds up squares of whole numbers starting from 1 until the total exceeds 10000.

Step 2: Turn this into a function `find_threshold()` that returns the number at which the threshold is exceeded.

Step 3: Modify to accept a parameter `threshold` (replacing the hardcoded 10000).

```
def find_threshold(threshold):
    # TODO Your code here
    pass

print(find_threshold(10000))
print(find_threshold(1000))
print(find_threshold(100))
```

Part c) Input Validation

Modify your `find_threshold()` function to:

1. Check if the input is an integer
2. Check if the input is positive
3. Print an error message and return None if invalid

```
def find_threshold(threshold):
    # TODO Add input validation here

    if not isinstance(threshold, int):
        print("Error: threshold must be an integer")
        return None

    if threshold <= 0:
```

```
            print("Error: threshold must be positive")
            return None

        # Original code here
        pass
```

Exercise 15: *Riemann Sum Function* (*)

Save as `ex15.py` Build progressively more sophisticated versions of a Riemann sum calculator.

Part a) Basic Riemann Sum Function

Convert your code from Exercise 11 into a function that:

- Calculates the Riemann sum for exp(x) from 0 to 4 with step 0.1
- Prints a text bar chart (one line of # characters per rectangle, as in Exercise 11 part a)
- Returns the calculated sum

```
import math

def riemann_sum():
    """Calculate Riemann sum for exp(x) from 0 to 4."""
    # TODO Your code here
    pass

result = riemann_sum()
print(f"Integral approximation: {result:.4f}")
```

Part b) Parametrized Boundaries

Modify your function to accept the lower and upper bounds as parameters:

```
def riemann_sum(a, b):
    """
    Calculate Riemann sum for exp(x) from a to b.

    Parameters:
    -----------
    a : float
        Lower bound of integration
    b : float
        Upper bound of integration
    """
    # TODO Your code here
    pass

# Test with different bounds:
print(riemann_sum(0, 4))
print(riemann_sum(1, 3))
print(riemann_sum(-1, 1))
```

Part c) Default Step Size

Add a `step` parameter with a default value of 0.1:

```
def riemann_sum(a, b, step=0.1):
    """
    Calculate Riemann sum for exp(x) from a to b.

    Parameters:
    -----------
    a : float
        Lower bound of integration
    b : float
        Upper bound of integration
    step : float, optional
        Integration step size (default: 0.1)
    """
    # TODO Your code here
    pass

# Test with different step sizes:
print(riemann_sum(0, 4))           # Uses default step
print(riemann_sum(0, 4, step=0.01))  # Finer steps
print(riemann_sum(0, 4, step=0.5))   # Coarser steps
```

Part d) Generic Function Integration (*)

Make your function work with any function, not just exp(x):

```
def riemann_sum(func, a, b, step=0.1):
    """
    Calculate Riemann sum for any function.

    Parameters:
    -----------
    func : function
        Function to integrate (must accept a single parameter)
    a : float
        Lower bound
    b : float
        Upper bound
    step : float, optional
        Step size (default: 0.1)
    """
    # TODO Your code here
    pass

# Define test functions:
def quadratic(x):
    """Calculate x² - 2x"""
    return x**2 - 2*x
```

```
        def exponential(x):
            """Calculate exp(x)"""
            return math.exp(x)

        # Test with different functions:
        print(riemann_sum(exponential, 0, 4))
        print(riemann_sum(quadratic, 0, 2))
        print(riemann_sum(math.sin, 0, math.pi))  # sin from 0 to π should be
~2
```

Exercise 16: *Neuroscience Functions*

Save as `ex16.py`

Create a collection of functions for common neuroscience calculations.

Part a) Firing Rate Calculator

```
        def calculate_firing_rate(spike_count, duration):
            """
            Calculate firing rate in Hz.

            Parameters:
            -----------
            spike_count : int
                Number of spikes observed
            duration : float
                Recording duration in seconds

            Returns:
            --------
            float
                Firing rate in Hz (spikes/second)
            """
            # TODO Your code here
            pass

        # Test:
        rate = calculate_firing_rate(150, 10.0)
        print(f"Firing rate: {rate} Hz")
```

Part b) Inter-Spike Interval Analysis

```
        def calculate_isi_statistics(spike_times):
            """
            Calculate inter-spike interval statistics.

            Parameters:
            -----------
```

```
    spike_times : list
        List of spike times in seconds

    Returns:
    --------
    dict
        Dictionary containing 'mean_isi', 'std_isi', and 'cv'
        (coefficient of variation = std/mean)
    """
    # TODO Your code here
    # 1. Calculate ISIs (differences between consecutive spike times)
    # 2. Calculate mean ISI
    # 3. Calculate standard deviation of ISI
    # 4. Calculate CV
    # 5. Return as dictionary
    pass

# Test:
spikes = [0.1, 0.3, 0.45, 0.7, 0.85, 1.1, 1.25]
stats = calculate_isi_statistics(spikes)
print(f"Mean ISI: {stats['mean_isi']:.3f} s")
print(f"CV: {stats['cv']:.3f}")
```

Part c) Voltage Classifier

```
def classify_membrane_state(voltage):
    """
    Classify membrane potential state.

    Parameters:
    -----------
    voltage : float
        Membrane potential in mV

    Returns:
    --------
    str
        State classification: 'hyperpolarized', 'resting',
        'depolarized', or 'action_potential'
    """
    # TODO Your code here
    # Use ranges:
    # hyperpolarized: < -80 mV
    # resting: -80 to -65 mV
    # depolarized: -65 to -30 mV
    # action_potential: > -30 mV
    pass

# Test:
for v in [-90, -70, -50, 0]:
```

```
    state = classify_membrane_state(v)
    print(f"{v} mV: {state}")
```

Part d) Batch Processing (*)

Write a function that processes multiple recordings:

```
def analyze_recordings(recordings):
    """
    Analyze multiple spike train recordings.

    Parameters:
    -----------
    recordings : list of dict
        Each dict contains 'spike_times' and 'duration'

    Returns:
    --------
    list of dict
        Analysis results for each recording
    """
    # TODO Your code here
    # For each recording:
    # 1. Calculate firing rate
    # 2. Calculate ISI statistics
    # 3. Store results
    pass

# Test:
data = [
    {'spike_times': [0.1, 0.3, 0.5, 0.9, 1.2], 'duration': 2.0},
    {'spike_times': [0.05, 0.15, 0.35, 0.55], 'duration': 1.0},
]

results = analyze_recordings(data)
for i, result in enumerate(results):
    print(f"Recording {i+1}: {result}")
```

Exercise 17: *Recursive Functions* (*)

Save as `ex17.py`

Recursion is when a function calls itself. Every recursive function needs:

1. A base case (when to stop)
2. A recursive case (calling itself with a simpler problem)

Part a) Factorial

Write a recursive function to calculate factorial:

```
def factorial(n):
    """
    Calculate n! recursively.

    n! = n × (n-1) × (n-2) × ... × 2 × 1

    Parameters:
    -----------
    n : int
        Non-negative integer

    Returns:
    --------
    int
        n!
    """
    # Base case: 0! = 1 and 1! = 1

    # Recursive case: n! = n × (n-1)!

    pass

# Test:
print(factorial(5))   # Should be 120
print(factorial(0))   # Should be 1
```

Part b) Fibonacci

Write a recursive function for the Fibonacci sequence:

```
def fibonacci(n):
    """
    Calculate the nth Fibonacci number recursively.

    Sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

    Parameters:
    -----------
    n : int
        Position in sequence (0-indexed)

    Returns:
    --------
    int
        nth Fibonacci number
    """
    # Base cases: F(0) = 0, F(1) = 1

    # Recursive case: F(n) = F(n-1) + F(n-2)

    pass
```

```
# Test:
for i in range(10):
    print(f"F({i}) = {fibonacci(i)}")
```

Part c) List Sum

Write a recursive function to sum all numbers in a list:

```
def recursive_sum(numbers):
    """
    Sum all numbers in a list recursively.

    Parameters:
    -----------
    numbers : list
        List of numbers

    Returns:
    --------
    float
        Sum of all numbers
    """
    # Base case: empty list sums to 0

    # Recursive case: sum = first element + sum of rest

    pass

# Test:
print(recursive_sum([1, 2, 3, 4, 5]))  # Should be 15
print(recursive_sum([]))                # Should be 0
```

Exercise 18: *Scope and Namespace* (*)

Save as `ex18.py`

Part a) Understanding Scope

Predict the output of this program, then run it:

```
x = "global"

def outer():
    x = "outer"

    def inner():
        x = "inner"
        print(f"Inner x: {x}")
```

```
        inner()
        print(f"Outer x: {x}")

    outer()
    print(f"Global x: {x}")
```

Part b) Modifying Global Variables

Fix this code so it correctly increments the counter:

```
    counter = 0

    def increment():
        # This doesn't work - why?
        counter = counter + 1

    increment()
    print(counter)  # Still 0!
```

Two solutions:

1. Use the `global` keyword (not recommended)
2. Return the new value and reassign (recommended)

Part c) Function Closures

Study this code and explain what happens:

```
    def make_multiplier(factor):
        """Create a function that multiplies by factor."""
        def multiply(x):
            return x * factor
        return multiply

    times_two = make_multiplier(2)
    times_ten = make_multiplier(10)

    print(times_two(5))   # ?
    print(times_ten(5))   # ?
```

Challenge Problems

Challenge 1: *Prime Number Generator*

Write a function that generates all prime numbers up to n using the Sieve of Eratosthenes algorithm. Include a progress indicator for large n.

Challenge 2: *Monte Carlo Pi Estimation*

Estimate π using the Monte Carlo method:

1. Generate random points in a unit square
2. Count how many fall inside a quarter circle
3. Use the ratio to estimate π

Challenge 3: Spike Train Simulation

Write a function that simulates a Poisson spike train given a firing rate and duration. Return a list of spike times.

> [!hint] **Tips for Debugging**
>
> 1. **Print statements are your friend**: Add `print()` statements to see variable values
> 2. **Test incrementally**: Don't write everything at once
> 3. **Use meaningful names**: `spike_count` is better than `sc` or `x`
> 4. **Check types**: Use `type()` to verify variable types
> 5. **Read error messages carefully**: They usually tell you exactly what's wrong
> 6. **Use the debugger**: Learn to use Python's `pdb` debugger
> 7. **Ask for help**: Discuss with classmates and instructors

---

Additional Resources

Python Documentation

- Built-in Functions: https://docs.python.org/3/library/functions.html
- String Methods: https://docs.python.org/3/library/stdtypes.html#string-methods
- List Methods: https://docs.python.org/3/tutorial/datastructures.html

Practice Platforms

- Python Tutor (visualize code execution): https://pythontutor.com/
- Practice Python: https://www.practicepython.org/
- Project Euler: https://projecteuler.net/

Standard Library

- The Python Tutorial: https://docs.python.org/3/tutorial/
- Python Standard Library Reference: https://docs.python.org/3/library/
- math module: https://docs.python.org/3/library/math.html

> **Good luck with the exercises! Remember: programming is learned by doing. Don't be afraid to experiment, make mistakes, and ask questions.**